FPT Software

Testing Foundation



Table of Contents

1 The 7 Principles

2 Testing Life Cycle

3 Test Design & Techniques

4 Bug Lifecycle

The 7 Principles



Testing shows presence of defects

Theory	Example
What Testing Does: • Confirms that defects exist. • Reduces the probability of undiscovered bugs.	**Scenario: Login Functionality** • 10 test cases covering valid & invalid inputs all PASS.
What Testing CANNOT Do: • Passing all test cases does not prove zero bugs exist. • Latent defects can still remain in rare conditions.	**Production Reality:** • Users experience app crashes when switching quickly between Wi-Fi and 4G during login.

Exhaustive testing is impossible

Theory	Example
• Testing all combinations of inputs and preconditions is impossible due to limited time and budget. • Attempting to test everything delays releases without adding real quality value.	**Scenario 1: Age Verification Field** • Input Range: Valid age is 18 to 60 (system accepts numbers 0 to 100). • Why Exhaustive Fails: Testing every single number from 0 to 100 takes hundreds of repetitive tests for just one field.

How Applies in Practice:

- **In Test Planning:** Apply Risk-based Testing to prioritize critical features over low-impact UI elements.
- **In Test Design:** Apply standard techniques (EP, BVA, Decision Table) to eliminate redundant test cases while maintaining high coverage.

The 7 Principles

Early testing

Theory	Example
Core Principle: • Testing should start as early as possible in the Software Development Life Cycle • Finding and fixing defects early prevents defects from cascading into code and architecture.	• **User Story:** "Users can enter the amount to transfer." • **Asks early:** "What are the minimum/maximum limits? Can users enter negative numbers or decimals?"

How Applies in Practice:

- **Perform Static Testing:** Review Requirements (SRS), User Stories, and Design wireframes without executing code.
- **Join Team Discussions:** Actively participate in Requirement Grooming / Refinement meetings to clarify ambiguities.

The 7 Principles

Defect clustering

Theory	Example
The 80/20 Rule: • A small number of modules usually contain most of the defects found (~80% of bugs are in ~20% of modules). **Why It Happens:** • High business complexity • Code is frequently modified • Developers may lack specific domain experience.	• **Stable Modules (Low bugs):** Login, User Profile → Rarely have issues. • **Bug Hotspots (80% bugs):** Promotion & Voucher Engine. Payment Gateway Integration.

How Applies in Practice:

- **Smart Resource Allocation:** Focus extra testing effort and design deeper test scenarios around hotspot modules instead of testing all modules equally.

The 7 Principles

Pesticide paradox

Theory	Example
The Core Concept: • Running the exact same test cases repeatedly will eventually stop finding new defects. • The software becomes "immune" to those specific tests. **Role of Old Tests:** • Essential for Regression Testing (verifying existing features still work).	**Scenario: Checkout Flow** • **Old Test Suite:** Linear path (Add item → Cart → COD) → Always 100% PASS. • **Production Bug:** User taps Back, adds voucher, and switches to E-wallet → Wrong total price. • **Why Missed:** Old suite never tested back-navigation or switching payment mid-flow.

How Applies in Practice:

- **Apply Exploratory Testing & Error Guessing:** Test freely based on real user habits to find bugs outside the predefined checklist.

Testing is context dependent

Theory	Example
The Core Concept: • Testing is done differently in different contexts. • No universal test strategy fits all projects.	**Context A: Banking / Fintech App** • Top Priority: Security, Data Integrity, Regulatory Compliance. **Context B: Mobile Gaming App** • Top Priority: Performance, UI/UX, Graphics smoothness.

Absence of errors fallacy

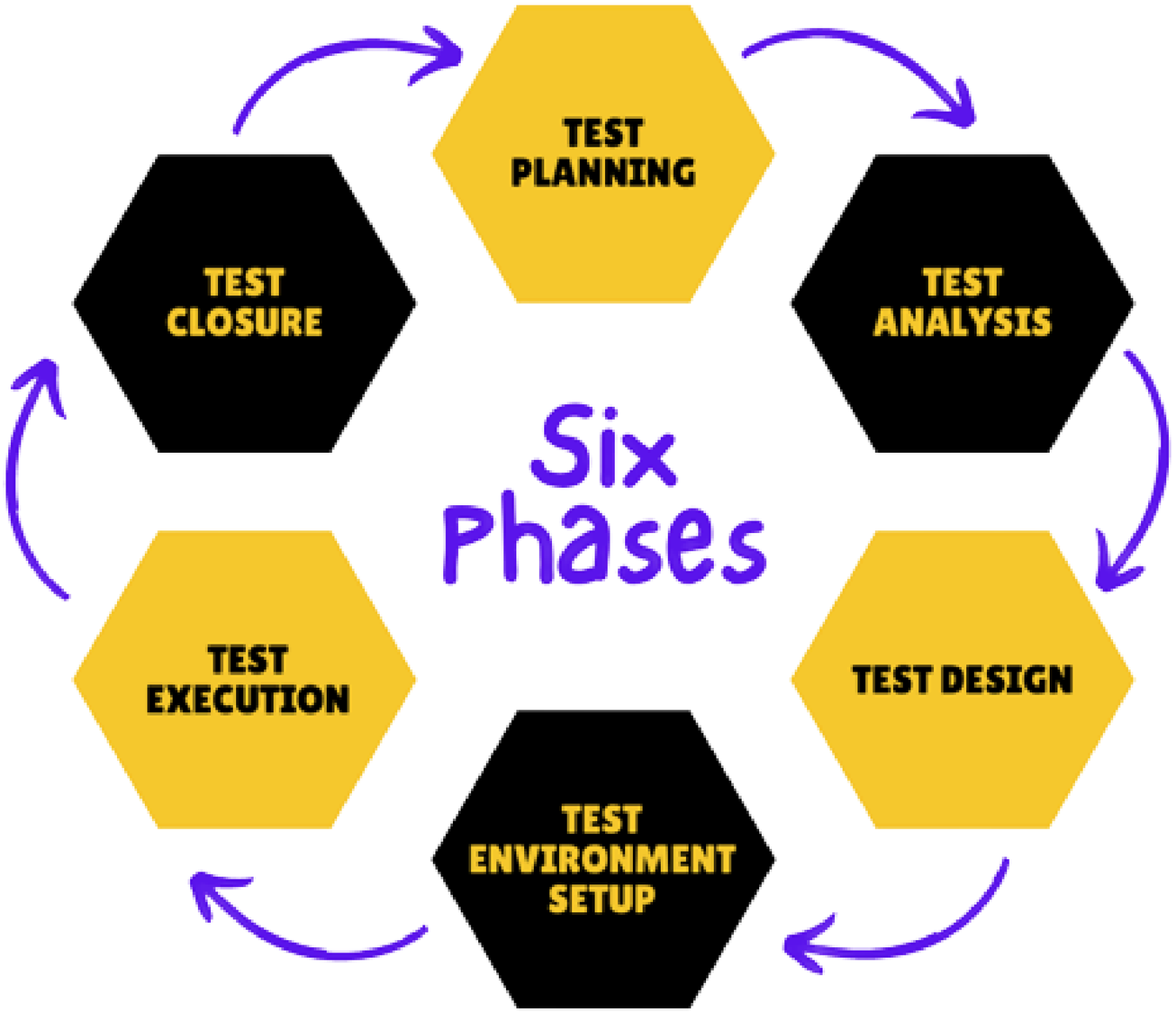
Example	Example
The Core Concept: • Finding and fixing all defects does not guarantee project success. • A bug-free system is still a failure if it does not fulfill user needs or business goals.	**Scenario: Ride-hailing App** • **Test Result:** Booking flow (driver matching, fare calculation) runs with 100% PASS (Zero bugs). • **Production Failure:** Users abandon the app because it lacks an in-app chat/call tool to locate drivers at crowded spots. • **Outcome:** Technically defect-free, but not fit for real-world use.

FPT Software

Testing Life Cycle



Software Testing Life Cycle



Software Testing Life Cycle - Test Planning

Inputs	Activities	Outputs	Owner
• **Requirements Base:** User Stories, SRS, Acceptance Criteria. • **Project Timeline:** Release dates, Sprint durations. • **Project Risks:** Initial business & technical risks.	• **Define Scope:** Clearly state what is In-scope and what is Out-of-scope. • **Set Criteria:** Establish Entry Criteria (when to start) and Exit Criteria / DoD (when to stop/ship). • **Resource & Schedule:** Assign testers to modules; estimate timeline & effort. • **Select Strategy:** Choose test levels, test types, and tools.	• **Test Plan:** Master Test Plan (Waterfall) or Sprint/Quick Plan (Agile). • **Estimation & Schedule:** Estimated effort and target milestones.	• Test Lead (with active participation of the QA team)

Software Testing Life Cycle - Test Analysis

Inputs	Activities	Outputs	Owner
• **Requirements:** SRS, User Stories, Acceptance Criteria. • **Design & Architecture:** DB Schema, System Architecture, Wireframes / Figma. • **Interfaces:** API specifications.	• **Review Test Basis:** Perform Static Testing to detect ambiguities, contradictions, or missing logic early. • **Identify Test Conditions:** Determine WHAT to test (verify payment with insufficient balance). • **Evaluate Testability:** Ensure requirements are clear and verifiable.	• **Test Conditions:** High-level list of verifiable aspects/scenarios. • **Defect Reports:** Ambiguities or bugs found directly from requirements (Static Testing).	• Test Engineer

Software Testing Life Cycle - Test Design

Inputs	Activities	Outputs	Owner
• **Test Conditions:** Generated from Test Analysis. • **Test Basis:** SRS, User Stories, UI/UX Mockups.	• **Apply Test Techniques:** ○ **Black-box:** EP, BVA, Decision Table, State Transition. ○ **White-box:** Statement, Branch Testing. ○ **Experience-based:** Error Guessing, Exploratory. • **Write Detailed Test Cases:** Define preconditions, steps, test data, and expected results. • **Identify Test Data & Env Needs:** Specify necessary accounts, payment cards, or test configurations.	• **Test Cases:** Low-level test cases ready for execution. • **Test Data Requirements:** Specification of data needed for tests.	• Test Engineer • **Reviewed by:** Senior QA / Test Lead

Software Testing Life Cycle - Test env

Inputs	Activities	Outputs	Owner
• **Test Plan:** Environment & hardware/software requirements. • **Application Build:** Deployable package from Dev team.	• **Hardware & OS Provisioning:** Set up servers, client machines, OS, and mobile devices. • **Software & Tools Setup:** Install required DB, test management/automation tools, and configure network/firewall. • **Deploy Application Build:** Deploy the latest code to Staging environment.	• **Ready Test Environment:** Stable Staging environment accessible to all testers.	• DevOps / System Admin

Software Testing Life Cycle - Test Execution

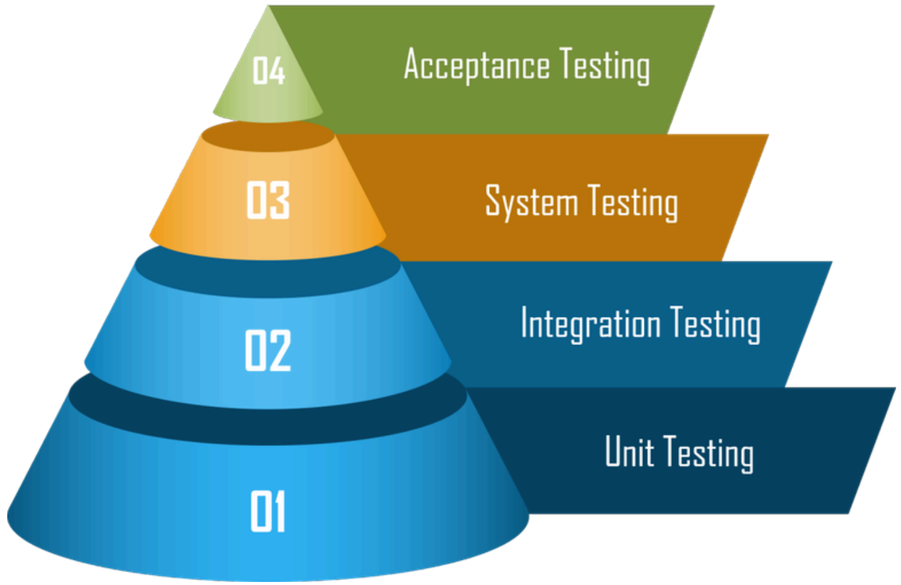
Inputs	Activities	Outputs	Owner
• **Test Deliverables:** Approved Test Cases, Test Data. • **Ready Environment:** Verified Staging environment.	• **Execute Test Cases:** Run manual and automated test scripts according to plan. • **Compare Results:** Verify Actual Result against Expected Result. • **Log Defects:** Report bugs in tracking tools with clear reproduction steps, screenshots/videos, and severity/priority.	• **Test Execution Results:** Status marked on all test cases (Pass / Fail / Blocked / Skipped). • **Defect Reports:** Documented bug tickets logged in tracking tools.	• Test Engineer

Software Testing Life Cycle - Test Closure

Inputs	Activities	Outputs	Owner
• **Test Deliverables:** All Test Execution Logs, Test Case results (Pass/Fail rate). • **Defect Status:** Defect Reports, resolved & outstanding/deferred bugs. • **Test Plan:** Baseline to verify Exit Criteria / DoD.	• **Evaluate Exit Criteria:** Verify whether planned exit criteria / Definition of Done (DoD) are fully met. • **Archive Test Assets:** Save test cases, automation scripts, and test data for future maintenance.	• **Test Summary Report:** Comprehensive quality report providing for Stakeholders.	• Test Lead • Supported by: All QA Team Members

Test Levels

Test Level	Focus Area	Core Objective
Unit Testing	Smallest parts of code (functions, methods)	Make sure code runs correctly alone
Integration Testing	How different parts talk to each other	Make sure modules work together
System Testing	The entire application from start to finish	Make sure the app meets all requirements
UAT	Real user scenarios	Make sure the app is ready for real users



Unit Testing

Dimension	Implementation Details
SDLC Phase & Basis	• Phase: Implementation (Coding) Phase • Test Basis: Low-Level Design, Class diagrams, Method logic
Owner	• Developer
Purpose	• Verify individual code units execute correctly in total isolation
Approach & Tools	• Technique: White-box Testing • Frameworks: xUnit, NUnit
Practical Example	• Method CalculateShippingFee(weight, distance): ○ Valid inputs (2kg, 5km) → Returns $2.50 ○ Negative weight (-1kg) → Throws InvalidWeightException

Integration Testing

Dimension	Implementation Details
SDLC Phase & Basis	• Phase: Integration Phase • Test Basis: System Architecture, API Specs, DB Schemas
Owner	• Tester
Purpose	• Validate data integrity and communication contracts between interconnected modules, services, and databases
Approach & Tools	• Techniques: Gray-box • Tools: Postman, Karate
Practical Example	• Order Service integrating with Payment Gateway API: ○ Gateway returns HTTP 200 + status 00 → DB status updates to PAID

System Testing

Dimension	Implementation Details
SDLC Phase & Basis	• Phase: System Verification Phase • Test Basis: Software Requirement Specification
Owner	• Tester
Purpose	• Evaluate the fully integrated end-to-end application against functional and non-functional requirements
Approach & Tools	• Technique: Black-box Testing (EP, BVA, Decision Table, State Transition) • Tools: Selenium, Playwright, ...
Practical Example	• E-Commerce E2E Flow: Customer registers → Verifies email → Applies voucher → Pays with Credit Card → System reduces DB stock → Admin verifies invoice

Acceptance Testing

Dimension	Implementation Details
SDLC Phase & Basis	• Phase: Acceptance Phase • Test Basis: Business Requirement Documents (BRD)
Owner	• Client, End-Users
Purpose	• Establish system confidence, prove operational readiness
Approach & Tools	• Black-box testing (Focus on real-world business flows) • Tools: Mostly manual testing
Practical Example	• Scenario: A real client uses the Beta version of an E-commerce app. • They try to buy a product and check if the process feels natural and if the final invoice is correct.

Where is Performance Testing executed?

Criteria	Integration Testing	System Testing
Scope	Single, isolated API.	Full, connected end-to-end system.
Test Flow	Send requests to one specific endpoint.	Simulate a complete User Journey (Login → Search → Pay).
Main Focus	Find slow code & Database queries early, before the UI is ready.	Check server stability & system crashes under heavy concurrent traffic.
Common Tools	JMeter, Postman	JMeter, K6

Test Types

Criteria	Functional Testing	Non-Functional Testing
Core Question	WHAT the system does	HOW WELL the system performs
Focus	Functions, features, business rules	Performance, security, usability, reliability
Based on	Functional requirements, User stories	Non-functional requirements
Examples	Smoke test, Regression test	Performance, Load, Stress, Security test
Example (Login)	Can the user log in with valid credentials?	Can the system handle 5,000 logins per second?
Which Test Level?	Executed at ALL levels (Unit, Integration, System, UAT).	Mainly at Integration and System levels.
Which STLC Phase?	Executed in the Test Execution phase.	Executed in the Test Execution phase (usually run after functional tests).

FPT Software

Test Design & Techniques



Test Scenario and Test Case

	Test Scenario	Test Case
STLC Phase	Test Analysis	Test Design
Core Question	WHAT to test?	HOW to test?
Level	High-level (overview summary)	Low-level (Step-by-step with data)
Main Goal	Ensure complete test coverage	Ensure accuracy and repeatability
Example (Login)	Verify user login functionality	• TC1: Login with valid username & password • TC2: Login with wrong password

Test Techniques

Black-Box Techniques

- **Basis**: Requirements & specifications.
- Code Access: No internal code visibility (outer behavior).
- Main Focus: Verifies WHAT the system does.
- Common Methods: Equivalence Partitioning, Boundary Value Analysis, Decision Tables.

White-Box Techniques

- **Basis**: Source code & system architecture.
- Code Access: Full internal logic and path visibility.
- Main Focus: Verifies HOW the code executes.
- Common Methods: Statement testing, branch coverage.

Experience-Based Techniques

- **Basis**: Tester intuition, domain skills & past defects.
- Code Access: Independent of specifications or code.
- Main Focus: Uncovers UNEXPECTED edge cases.
- Common Methods: Error guessing, exploratory testing, checklists.

Black-Box Techniques

Technique	Core Principle	Example
Equivalence Partitioning (EP)	Groups input data into valid and invalid sets. Test one value to represent the whole set.	Age range [18–60]: • 10 (Invalid) • 25 (Valid) • 70 (Invalid)
Boundary Value Analysis (BVA)	Tests the exact edges (min/max) of valid and invalid sets, where bugs happen most.	Input [18 to 60]: • Test edges: 17, 18, 19 and 59, 60, 61
Decision Table Testing	Combines multiple rules (IF-THEN conditions) into a clear matrix to check all outcomes.	
State Transition Testing	Tests how an object changes states when an event happens (both valid & invalid flows).	Order: Pending → Paid → Shipped Negative: Pending → Delivered (Blocked)

Black-Box Techniques

Requirement: x ≤ 10 return Valid

Bug: if (x == 10) Valid else Invalid

Technique	Test Values	Test Execution & Expected vs Actual	Result
2-Value BVA	x = 10 x = 11	• x = 10 Valid → Pass • x = 11 Invalid → Pass	MISSED BUG
3-Value BVA	x = 9 x=10 x=11	• x = 10 Valid → Pass • x = 11 Invalid → Pass • x = 9 expected Valid → Fail	BUG DETECTED

Black-Box Techniques

Discount Policy:

The system supports 3 payment methods: Cash, Credit Card, and E-wallet.

- **Gold members**: Always get a 15% discount.
- **Silver members**: Get 10% off if paying by E-wallet or ordering >= 500k; otherwise (Cash/Credit Card and under 500k), get 5% off.
- **Bronze members**: Only get 5% off if paying by E-wallet AND ordering >= 500k; otherwise, no discount (0%).

Black-Box Techniques

Member Tier	Gold	Gold	Gold	Gold	Gold	Gold	Silver	Silver	Silver	Silver	Silver	Silver	Bronze	Bronze	Bronze	Bronze	Bronze	Bronze
Payment	Cash	Cash	Credit	Credit	Wallet	Wallet	Cash	Cash	Credit	Credit	Wallet	Wallet	Cash	Cash	Credit	Credit	Wallet	Wallet
>= 500K	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N
15% Discount	X	X	X	X	X	X												
10% Discount							X		X		X	X						
5% Discount								X		X							X	
No Discount													X	X	X	X		X

Black-Box Techniques

Member Tier	Gold	Silver	Silver	Silver	Bronze	Bronze	Bronze
Payment	-	-	Cash / Credit	Wallet	Cash / Credit	Wallet	Wallet
>= 500K	-	Y	N	N	-	Y	N
15% Discount	X						
10% Discount		X		X			
5% Discount			X			X	
No Discount					X		X

White-Box Techniques

Technique	Core Principle	Example

Statement testing	Executes every single line of code at least once.	int total = a + b;
		print(total);
		→ 1 test case runs both lines (100% coverage)

Branch testing	Executes every decision outcome (both True and False paths).	if (score >= 50) pass(); else fail(); • Test 1: score = 60 (True path) • Test 2: score = 40 (False path)

White-Box Techniques

State Transition Rules:

- ACTIVE ➔ BLOCKED: Withdraw > 50M (Fraud risk) ➔ Lock card.
- ACTIVE ➔ ACTIVE: Withdraw ≤ 50M ➔ Proceed normally.
- BLOCKED ➔ BLOCKED: Card already locked ➔ Decline all.

```
function thucHienGiaoDich(trangThaiHienTai, soTien) {
    let trangThaiMoi = trangThaiHienTai;      // Lệnh 1: Mặc định giữ nguyên trạng
    let ketQua = "Từ chối giao dịch";         // Lệnh 2: Mặc định từ chối

    // Nhánh Quyết định 1
    if (trangThaiHienTai === "ACTIVE") {      // Lệnh 3
        ketQua = "Giao dịch thành công";      // Lệnh 4

        // Nhánh Quyết định 2 (Bên trong nhánh 1)
        if (soTien > 50000000) {              // Lệnh 5
            trangThaiMoi = "BLOCKED";         // Lệnh 6
        }
    }

    luuVaoDatabase(trangThaiMoi);             // Lệnh 7: Cập nhật trạng thái
    return ketQua;                            // Lệnh 8
}
```

Technique	Statement Coverage	Branch Coverage
Objective	Execute every line of code at least once.	Test every True/False outcome of decisions.
Min. Test Cases	• ACTIVE, amount = 60	• ACTIVE, amount = 60 • ACTIVE, amount = 10
Risk	100% coverage achieved, but completely misses the BLOCKED state behavior.	Forces testing of both state-changing (True) and non-changing (False) paths.

White-Box Techniques

Why use Statement Testing if Branch is better?

Theory	Reality
• 100% Branch Coverage automatically guarantees 100% Statement Coverage. • If we have unlimited time, Branch Testing is all we need.	• Branch testing requires significantly more test cases. • In massive projects, achieving 100% Branch coverage is too expensive and time-consuming. • Statement: Used for simple, low-risk modules to save time. • Branch: Reserved for complex, high-risk logic

Experience-Based Techniques

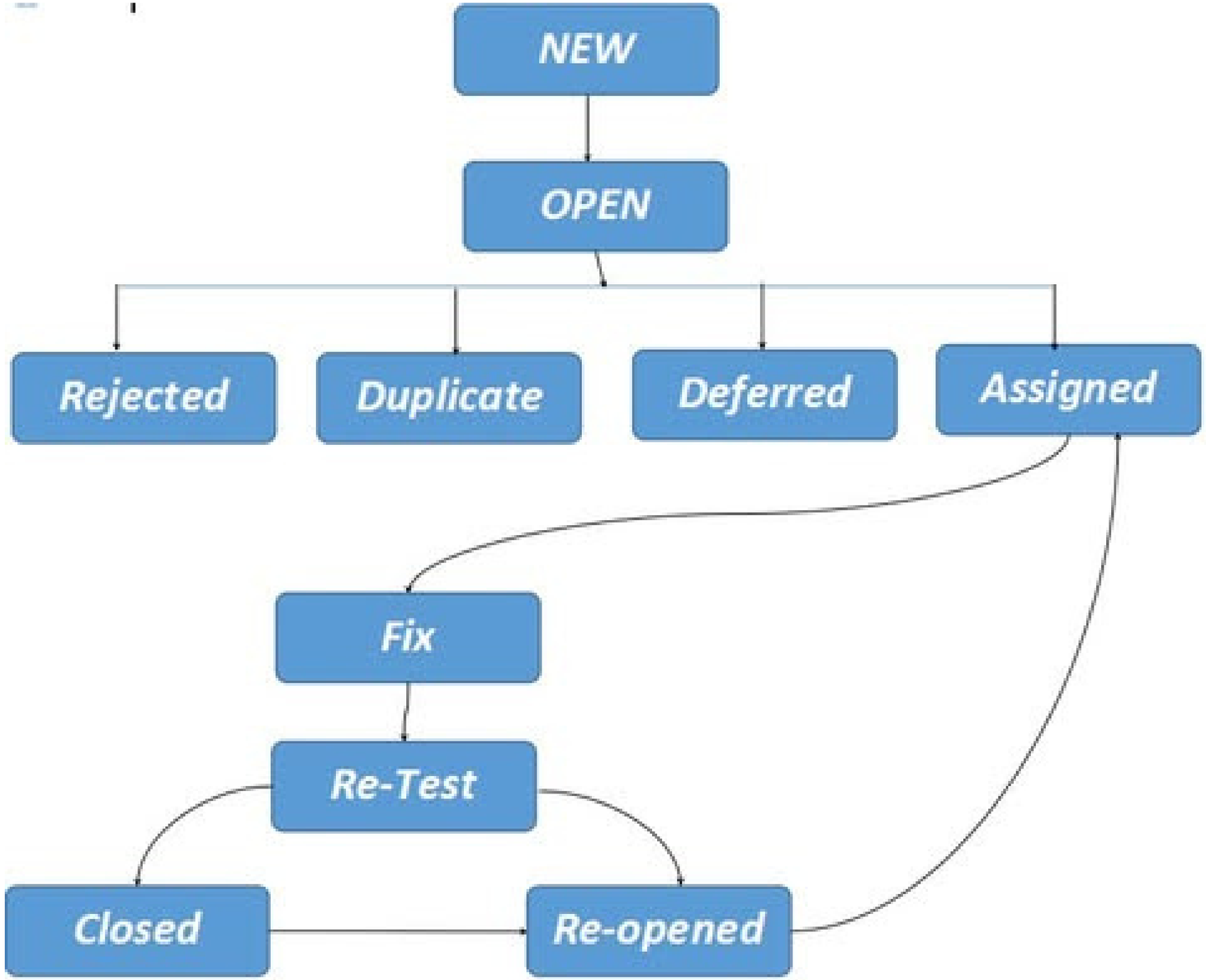
Technique	Core Principle	Example
Error Guessing	Predicts where bugs might hide based on past experience and intuition.	Test blank fields, negative numbers, or spamming the "Submit" button.
Exploratory Testing	Tests and learns the app freely without pre-written test cases.	Spend 30 minutes exploring the Checkout flow on poor Wi-Fi.
Checklist-Based Testing	Tests against a pre-made list of common items or rules.	Follow a UI checklist: check fonts, button colors, and responsive views.

FPT Software

Bug Lifecycle



Bug/Defect Lifecycle




THANK YOU!